CẨM NANG SONG NGỮ VIỆT - TRUNG / 越中双语指南

LỘ TRÌNH TRIỂN KHAI MÔ HÌNH QWEN

QWEN 模型实施路线图

Từ hạ tầng và dữ liệu đến QLoRA, Next.js và production

从基础设施与数据到 QLoRA、Next.js 和生产部署

Dành cho người mới triển khai LLM self-host / 面向首次自建部署 LLM 的学习者

Phạm vi **范围**	**Đối tượng** **适用对象**	**Mục tiêu** **目标**
Qwen cho ứng dụng Việt-Anh 面向越英应用的 Qwen	Lập trình viên mới với AI 初次接触 AI 的开发者	Self-host, benchmark và production 自建部署、基准测试与生产上线

Phiên bản 1.0 - Tháng 8 năm 2026 / 1.0 版 - 2026 年 8 月

MỤC LỤC / 目录

1. Tìm hiểu model, tham số và chi phí hạ tầng 1. 了解模型、参数与基础设施成本 4

1.1. Kiến thức về model 1.1. 模型基础知识 4

1.2. Cách ước lượng VRAM 1.2. VRAM 估算方法 4

1.3. Chi phí phải tính 1.3. 需要核算的成本 5

2. Xác định kiến thức cần học và phần có thể bỏ qua 2. 明确必学知识与可暂缓内容 6

2.1. Linux 2.1. Linux 6

2.2. Docker và GPU container 2.2. Docker 与 GPU 容器 6

2.3. Chạy model Hugging Face bằng Python 2.3. 使用 Python 运行 Hugging Face 模型 6

2.4. Những gì có thể tạm bỏ qua 2.4. 可暂时不学的内容 7

3. Xây kho dữ liệu Việt-Anh có kiểm duyệt 3. 建设经审核的越英数据集 7

3.1. File hướng dẫn gán nhãn 3.1. 标注指南文件 7

3.2. Quy trình dữ liệu 3.2. 数据处理流程 8

3.3. Dữ liệu trẻ em 3.3. 儿童数据 8

4. Chạy Qwen self-host và benchmark trước khi train 4. 在训练前自建部署 Qwen 并进行基准测试 9

4.1. Kiến trúc pilot 4.1. 试点架构 9

4.2. Dựng model server 4.2. 搭建模型服务器 9

4.3. Benchmark trước khi train 4.3. 训练前基准测试 10

5. Fine-tune bằng SFT + QLoRA 5. 使用 SFT + QLoRA 进行微调 10

5.1. Cơ chế fine-tune 5.1. 微调机制 10

5.2. Định dạng hội thoại 5.2. 对话数据格式 11

5.3. Quy trình train 5.3. 训练流程 11

6. Tích hợp backend Next.js và triển khai production 6. 集成 Next.js 后端并部署到生产环境 12

6.1. Thiết kế provider 6.1. Provider 设计 12

6.2. Cấu hình server-side 6.2. 服务端配置 12

6.3. Fallback - luồng xử lý 6.3. Fallback 回退流程 13

6.4. Triển khai 6.4. 上线部署 13

7. Thứ tự thực hiện đề xuất 7. 建议实施顺序 14

1. Tìm hiểu model, tham số và chi phí hạ tầng

1. 了解模型、参数与基础设施成本

Mục tiêu của giai đoạn này là hiểu model cần bao nhiêu bộ nhớ, phục vụ được bao nhiêu request và tổng chi phí vận hành thực tế.

本阶段的目标是弄清模型需要多少显存、能够处理多少并发请求，以及实际运营总成本。

1.1. Kiến thức về model

1.1. 模型基础知识

- Phân biệt Base model và Instruct model. Ứng dụng hội thoại hoặc dịch nên bắt đầu từ bản Instruct.
  区分基础模型和指令模型。对话或翻译应用应优先从指令模型开始。
- Hiểu token và tokenizer: model xử lý chuỗi token số, không đọc trực tiếp từ hoặc câu.
  理解 Token 与分词器：模型处理的是数字 Token 序列，而不是直接读取单词或句子。
- Hiểu context length: tổng token đầu vào và đầu ra mà một request có thể sử dụng.
  理解上下文长度：一次请求可使用的输入与输出 Token 总量。
- Phân biệt FP32, BF16/FP16, INT8 và INT4; precision càng thấp thường càng tiết kiệm VRAM.
  区分 FP32、BF16/FP16、INT8 和 INT4；精度越低通常越节省显存。
- Phân biệt inference và training. Training phải lưu activation, gradient và optimizer state nên tốn bộ nhớ hơn nhiều.
  区分推理与训练。训练还需保存激活值、梯度和优化器状态，因此显存消耗高得多。
- Theo dõi latency, token/giây, throughput, concurrency, KV cache và tỷ lệ OOM.
  关注延迟、每秒 Token 数、吞吐量、并发数、KV Cache 与 OOM 比例。

1.2. Cách ước lượng VRAM

1.2. VRAM 估算方法

Bộ nhớ thô của trọng số = số tham số x số byte cho mỗi tham số. Với model 4B, phép tính chỉ cho biết trọng số; runtime còn cần KV cache, CUDA và tensor trung gian.

权重的理论显存 = 参数数量 × 每个参数占用的字节数。对于 4B 模型，该计算仅涵盖权重；运行时还需要 KV Cache、CUDA 和中间张量。

Định dạng 格式	Trọng số thô model 4B 4B 模型权重理论占用	Ghi chú 说明
FP32 FP32	Khoảng 16 GB 约 16 GB	Không phù hợp để tiết kiệm VRAM 不适合节省显存

Định dạng 格式	Trọng số thô model 4B 4B 模型权重理论占用	Ghi chú 说明
BF16 / FP16 BF16 / FP16	Khoảng 8 GB 约 8 GB	Phổ biến khi train/inference chất lượng cao 常用于高质量训练或推理
INT8 INT8	Khoảng 4 GB 约 4 GB	Giảm bộ nhớ 降低显存占用
INT4 INT4	Khoảng 2 GB 约 2 GB	Phù hợp thử nghiệm và QLoRA 适合测试与 QLoRA

> **Lưu ý thực tế:** Không dùng con số trọng số thô để chọn GPU. Phải cộng thêm KV cache, context length, concurrency, CUDA runtime và khoảng dự phòng chống OOM.
> **实际注意：**不能只根据理论权重选择 GPU，还必须考虑 KV Cache、上下文长度、并发数、CUDA 运行时以及防止 OOM 的预留空间。

1.3. Chi phí phải tính

1.3. 需要核算的成本

Không chỉ có tiền GPU. Cần xây bảng tổng chi phí sở hữu và vận hành theo tháng.
成本不仅是 GPU 费用，还应建立按月计算的总体拥有成本与运营成本表。

- Máy chủ CPU, RAM và SSD.
  服务器 CPU、内存与 SSD。
- Dung lượng tải model và lưu checkpoint.
  下载模型与保存检查点所需的存储空间。
- Điện, làm mát hoặc giá thuê GPU theo giờ.
  电力、散热或按小时租用 GPU 的费用。
- Băng thông mạng và lưu lượng truyền dữ liệu.
  网络带宽与数据传输流量。
- Monitoring, cảnh báo và lưu log.
  监控、告警与日志存储。
- Backup dataset, adapter, model card và training config.
  备份数据集、Adapter、模型卡与训练配置。
- Thời gian kỹ sư triển khai, trực vận hành và xử lý sự cố.
  工程师部署、值班运维与故障处理时间。
- GPU dự phòng hoặc kế hoạch fallback khi máy chính lỗi.
  备用 GPU，或主机故障时的回退方案。

> **Đầu ra:** Bảng capacity planning gồm GPU/VRAM, context, concurrency, P50/P95, token/giây, OOM và chi phí trên 1.000 request.
> **交付物：**容量规划表：GPU/显存、上下文长度、并发数、P50/P95、每秒 Token 数、OOM 率以及每 1,000 次请求的成本。

2. Xác định kiến thức cần học và phần có thể bỏ qua
2. 明确必学知识与可暂缓内容

2.1. Linux
2.1. Linux

- Thao tác file, thư mục, quyền truy cập, process và biến môi trường.
  掌握文件、目录、权限、进程和环境变量操作。
- Đọc log, kiểm tra port, dung lượng ổ đĩa, RAM và tiến trình đang chạy.
  查看日志、端口、磁盘空间、内存与运行中的进程。
- Dùng nvidia-smi để kiểm tra GPU, VRAM, nhiệt độ và process sử dụng GPU.
  使用 nvidia-smi 检查 GPU、显存、温度以及占用 GPU 的进程。
- Biết SSH, systemd hoặc dịch vụ tương đương để vận hành model server.
  掌握 SSH、systemd 或等效服务，用于运行模型服务器。

2.2. Docker và GPU container
2.2. Docker 与 GPU 容器

- Hiểu image, container, Dockerfile, volume, port, network và health check.
  理解镜像、容器、Dockerfile、Volume、端口、网络与健康检查。
- Cài NVIDIA driver và NVIDIA Container Toolkit đúng phiên bản.
  安装版本匹配的 NVIDIA 驱动与 NVIDIA Container Toolkit。
- Chạy container với GPU và kiểm tra torch.cuda.is_available().
  使用 GPU 启动容器，并检查 torch.cuda.is_available()。
- Mount cache/model vào volume để không tải lại model mỗi lần tạo container.
  将缓存和模型挂载到 Volume，避免每次创建容器都重新下载。

2.3. Chạy model Hugging Face bằng Python
2.3. 使用 Python 运行 Hugging Face 模型

- Tạo virtual environment và khóa phiên bản transformers, torch, tokenizer.
  创建虚拟环境，并锁定 transformers、torch 与 tokenizer 的版本。

- Tải tokenizer và model; hiểu torch_dtype, device_map và quantization config.
  加载分词器与模型，并理解 torch_dtype、device_map 与量化配置。
- Áp dụng đúng chat template và kiểm soát max_new_tokens, temperature, top_p.
  正确应用聊天模板，并控制 max_new_tokens、temperature 与 top_p。
- Đo thời gian tải model, VRAM sau khi tải và tốc độ sinh token.
  测量模型加载时间、加载后的显存占用与生成速度。

2.4. Những gì có thể tạm bỏ qua
2.4. 可暂时不学的内容

- Pretraining model và tokenizer từ đầu.
  从零开始预训练模型与分词器。
- Tự viết CUDA kernel hoặc tối ưu compiler.
  自行编写 CUDA Kernel 或优化编译器。
- Distributed training nhiều node, NVLink và InfiniBand.
  多节点分布式训练、NVLink 与 InfiniBand。
- RLHF, reward model, DPO hoặc GRPO.
  RLHF、奖励模型、DPO 或 GRPO。

> **Đầu ra:** Chạy được Qwen bằng Python và dựng được một Docker container sử dụng GPU, có log và health check cơ bản.
> **交付物：**能够使用 Python 运行 Qwen，并搭建可使用 GPU、具备基本日志与健康检查的 Docker 容器。

3. Xây kho dữ liệu Việt-Anh có kiểm duyệt
3. 建设经审核的越英数据集

3.1. File hướng dẫn gán nhãn
3.1. 标注指南文件

Guideline phải đủ rõ để hai người khác nhau tạo ra nhãn tương đương. Mỗi thay đổi cần có version và ngày hiệu lực.
标注指南必须足够清晰，使不同标注人员能够给出一致结果。每次修改都应记录版本与生效日期。

- Mục tiêu dataset và phạm vi câu được chấp nhận.
  数据集目标与可接受句子的范围。
- Schema bắt buộc: id, câu Việt, bản dịch Anh, ngữ cảnh, vùng miền, reviewer và trạng thái duyệt.
  必填 Schema：ID、越南语句子、英语译文、语境、地区、审核人和审核状态。

- Quy tắc dịch phủ định, chủ thể, người nghe, màu sắc, số lượng, thời gian và địa điểm.
  关于否定、主语、听话人、颜色、数量、时间和地点的翻译规则。
- Cách xử lý câu thiếu, nói lắp, từ địa phương và câu chưa hoàn chỉnh.
  处理残句、口吃、方言和未完成句子的方式。
- Tiêu chí loại bỏ mẫu sai, nhiễu, trùng lặp hoặc không có quyền sử dụng.
  剔除错误、噪声、重复或无使用授权样本的标准。
- Quy trình người gán nhãn, người review và người phân xử bất đồng.
  标注员、审核员以及争议裁决人的工作流程。

3.2. Quy trình dữ liệu
3.2. 数据处理流程

1. Thu thập dữ liệu và ghi nguồn/consent.
   收集数据，并记录来源与授权信息。
2. Ẩn danh và loại thông tin nhận dạng cá nhân.
   匿名化并删除个人身份信息。
3. Gán nhãn theo guideline.
   按照标注指南进行标注。
4. Review độc lập và xử lý bất đồng.
   独立审核并处理分歧。
5. Deduplicate và kiểm tra chất lượng.
   去重并进行质量检查。
6. Chia train/validation/test theo người nói hoặc cụm ngữ nghĩa.
   按说话人或语义簇划分训练、验证与测试集。
7. Version dataset và khóa test set.
   对数据集进行版本管理并锁定测试集。

3.3. Dữ liệu trẻ em
3.3. 儿童数据

- Có sự đồng ý phù hợp của phụ huynh hoặc người giám hộ.
  取得家长或监护人的适当同意。
- Nêu rõ mục đích sử dụng, thời gian lưu trữ và quyền yêu cầu xóa.
  明确使用目的、保存期限以及删除请求权。
- Dùng speaker_id ẩn danh; không dùng tên thật trong dataset.
  使用匿名 speaker_id，不在数据集中保存真实姓名。

- Loại số điện thoại, địa chỉ, trường học, tài khoản và thông tin nhận dạng.
  删除电话号码、地址、学校、账号及其他身份信息。
- Mã hóa dữ liệu, phân quyền tối thiểu và ghi audit log.
  加密数据、实施最小权限并记录审计日志。
- Không đưa dataset riêng tư lên Hugging Face Hub hoặc kho công khai.
  不得将私有数据集上传到 Hugging Face Hub 或公开仓库。

> **Đầu ra:** Guideline có version, dataset đã ẩn danh, golden test được khóa và hồ sơ consent/nguồn dữ liệu.
> **交付物：**具备版本号的标注指南、已匿名化的数据集、锁定的 Golden Test，以及授权与数据来源记录。

4. Chạy Qwen self-host và benchmark trước khi train
4. 在训练前自建部署 Qwen 并进行基准测试

4.1. Kiến trúc pilot
4.1. 试点架构

```
Ứng dụng / 应用
  ↓ HTTPS + API key
Next.js backend / Next.js 后端
  ↓ OpenAI-compatible API
Qwen model server / Qwen 模型服务器
  ↓
vLLM + GPU + Qwen
```

- Model chạy trên GPU service riêng, không nạp trực tiếp vào tiến trình Next.js.
  模型运行在独立 GPU 服务中，不直接加载进 Next.js 进程。
- Next.js chỉ gửi request, kiểm tra response và thực hiện fallback.
  Next.js 只负责发送请求、校验响应与执行回退。
- ASR, LLM và TTS nên là các service độc lập để dễ mở rộng và xử lý lỗi.
  ASR、LLM 与 TTS 应拆分为独立服务，便于扩展和故障处理。

4.2. Dựng model server
4.2. 搭建模型服务器

1. Chuẩn bị Linux GPU host, NVIDIA driver và Container Toolkit.
   准备 Linux GPU 主机、NVIDIA 驱动与 Container Toolkit。

2. Dựng vLLM và khóa version model, CUDA, PyTorch và Docker image.
   部署 vLLM，并锁定模型、CUDA、PyTorch 与 Docker 镜像版本。
3. Khởi chạy OpenAI-compatible endpoint /v1/chat/completions.
   启动兼容 OpenAI 的 /v1/chat/completions 接口。
4. Giới hạn context, max_tokens, concurrency và GPU memory utilization.
   限制上下文长度、max_tokens、并发数与 GPU 显存利用率。
5. Bảo vệ bằng HTTPS, API key, firewall và allowlist.
   通过 HTTPS、API Key、防火墙与白名单保护服务。
6. Thêm health check, readiness, log, metrics và cảnh báo OOM.
   增加健康检查、就绪检查、日志、指标与 OOM 告警。

4.3. Benchmark trước khi train

4.3. 训练前基准测试

- Dùng cùng prompt và golden test để so Cloudflare, Qwen gốc và Qwen đã tối ưu prompt.
  使用相同 Prompt 与 Golden Test，对比 Cloudflare、原始 Qwen 与优化 Prompt 后的 Qwen。
- Đo fidelity, lỗi phủ định, format, human acceptance, P50/P95, token/giây, timeout và OOM.
  测量忠实度、否定错误、格式、人工接受率、P50/P95、每秒 Token 数、超时与 OOM。
- Chỉ fine-tune khi lỗi lặp lại và prompt/rule không giải quyết ổn định.
  仅当错误反复出现且 Prompt/规则无法稳定解决时，才进行微调。

> **Đầu ra:** Model server gọi được qua API, có báo cáo benchmark và quyết định go/no-go cho fine-tune.
> **交付物：**模型服务器可通过 API 调用，并形成基准测试报告及是否进行微调的 Go/No-Go 决策。

5. Fine-tune bằng SFT + QLoRA

5. 使用 SFT + QLoRA 进行微调

5.1. Cơ chế fine-tune

5.1. 微调机制

```
Base model 4-bit, đóng băng / 4-bit 基础模型，冻结
            +
LoRA adapter được train / 训练 LoRA Adapter
            ↓
Base model + adapter / 基础模型 + Adapter
```

- SFT học từ cặp đầu vào và câu trả lời chuẩn do con người kiểm duyệt.
  SFT 从经人工审核的输入与标准回答对中学习。
- LoRA giữ nguyên trọng số gốc và chỉ train các ma trận low-rank nhỏ.
  LoRA 冻结原始权重，只训练小型低秩矩阵。
- QLoRA nạp base model ở 4-bit để giảm VRAM nhưng train adapter ở precision cao hơn.
  QLoRA 以 4-bit 加载基础模型以降低显存，同时使用更高精度训练 Adapter。
- Adapter phải có version, training config, dataset manifest và model card.
  Adapter 必须配套版本号、训练配置、数据集清单与模型卡。

5.2. Định dạng hội thoại

5.2. 对话数据格式

```
{
 "messages": [
  {"role": "system", "content": "Translate Vietnamese to English faithfully."},
  {"role": "user", "content": "Con không muốn đi học."},
  {"role": "assistant", "content": "I don't want to go to school."}
 ]
}
```

- System định nghĩa nhiệm vụ và quy tắc output.
  System 定义任务与输出规则。
- User chứa câu tiếng Việt cần xử lý.
  User 包含需要处理的越南语句子。
- Assistant chứa bản dịch tiếng Anh đã được kiểm duyệt.
  Assistant 包含经审核的英语译文。
- Ưu tiên tính loss trên phần assistant/completion và kiểm tra đúng chat template.
  优先仅对 Assistant/Completion 部分计算 Loss，并确认聊天模板正确。

5.3. Quy trình train

5.3. 训练流程

1. Khóa version model, tokenizer, dataset và config.
   锁定模型、分词器、数据集与配置版本。
2. Chạy baseline trên test set trước khi train.
   训练前先在测试集上运行基线。
3. Train adapter và theo dõi training/validation loss.
   训练 Adapter，并监控训练与验证 Loss。

4. Chọn checkpoint bằng validation, sau đó đánh giá test set kín.
   根据验证集选择检查点，再评估封闭测试集。
5. So sánh model gốc và adapter theo từng nhóm lỗi.
   按错误类别对比原始模型与 Adapter。
6. Đóng gói adapter, model card và báo cáo kết quả.
   打包 Adapter、模型卡与结果报告。

> **Đầu ra:** LoRA adapter, training config, biểu đồ loss và báo cáo chứng minh model tốt hơn baseline mà không overfit.
> **交付物：** LoRA Adapter、训练配置、Loss 曲线，以及证明模型优于基线且未过拟合的评估报告。

6. Tích hợp backend Next.js và triển khai production
6. 集成 Next.js 后端并部署到生产环境

6.1. Thiết kế provider
6.1. Provider 设计

```
translateVietnameseToEnglish(input, options)
  ├─ cloudflare
  └─ self_hosted → OpenAI-compatible API
```

- Tạo interface chung để nghiệp vụ không phụ thuộc trực tiếp vào vLLM.
  创建统一接口，使业务逻辑不直接依赖 vLLM。
- Tách provider, prompt builder, response validator và telemetry.
  拆分 Provider、Prompt Builder、响应校验与遥测模块。
- Dùng feature flag để chuyển provider hoặc rollback mà không sửa toàn bộ luồng.
  通过 Feature Flag 切换 Provider 或回滚，无需修改整条业务流程。

6.2. Cấu hình server-side
6.2. 服务端配置

```
TEXT_AI_PROVIDER=self_hosted
SELF_HOSTED_LLM_BASE_URL=https://gpu.example.com/v1
SELF_HOSTED_LLM_API_KEY=<secret>
SELF_HOSTED_LLM_MODEL=Qwen/Qwen3-4B-Instruct-2507
SELF_HOSTED_LLM_TIMEOUT_MS=2500
```

- API key chỉ tồn tại server-side; không dùng tiền tố NEXT_PUBLIC_.
  API Key 只能存在于服务端，不得使用 NEXT_PUBLIC_ 前缀。
- Giới hạn input, max_tokens, timeout, request size và concurrency.
  限制输入长度、max_tokens、超时、请求大小与并发数。

- Ghi request ID, model version, adapter version, latency và fallback reason.
  记录请求 ID、模型版本、Adapter 版本、延迟与回退原因。

6.3. Fallback - luồng xử lý

6.3. Fallback 回退流程

```
Cache
 ↓ miss
Qwen self-host
 ↓ timeout / OOM / response invalid
Cloudflare fallback
 ↓
Normalize + return
```

- Ưu tiên cache để giảm chi phí và latency.
  优先使用缓存以降低成本与延迟。
- Qwen trả kết quả phải qua schema và fidelity validator.
  Qwen 的结果必须通过 Schema 与忠实度校验。
- Khi timeout, OOM hoặc output lỗi, chuyển về Cloudflare có kiểm soát.
  发生超时、OOM 或输出异常时，受控回退到 Cloudflare。
- Dùng circuit breaker để tránh dồn request vào GPU đang lỗi.
  使用熔断器，避免将请求持续发送到故障 GPU。

6.4. Triển khai

6.4. 上线部署

1. Offline benchmark trên golden test.
   在 Golden Test 上进行离线基准测试。
2. Shadow traffic: Qwen chạy song song, Cloudflare vẫn trả kết quả.
   影子流量：Qwen 并行运行，但仍由 Cloudflare 返回结果。
3. Canary 5% và giám sát chất lượng, P95, timeout, OOM.
   先灰度 5%，监控质量、P95、超时与 OOM。
4. Tăng dần 25%, 50%, 100% khi đạt ngưỡng.
   达到指标后逐步提升至 25%、50% 和 100%。
5. Giữ rollback model/adapter và Cloudflare fallback.
   始终保留模型/Adapter 回滚能力与 Cloudflare 回退。

> **Đầu ra:** Provider self-host trong Next.js, cấu hình server-side an toàn, fallback hoạt động, dashboard và quy trình shadow/canary/rollback.

> **交付物：** Next.js 中的自建 Provider、安全的服务端配置、可用的回退流程、监控看板，以及影子流量、灰度发布与回滚流程。

7. Thứ tự thực hiện đề xuất

7. 建议实施顺序

Giai đoạn 阶段	Công việc 工作内容	Đầu ra 交付物
1 1	Model, VRAM và chi phí 模型、显存与成本	Capacity planning 容量规划
2 2	Linux, Python và Docker GPU Linux、Python 与 GPU 容器	Model chạy được 模型可运行
3 3	Guideline và dữ liệu 标注指南与数据	Golden test Golden Test
4 4	Self-host và benchmark 自建部署与基准测试	Go/No-Go fine-tune 微调 Go/No-Go
5 5	SFT + QLoRA SFT + QLoRA	Adapter đã đánh giá 已评估 Adapter
6 6	Next.js và production Next.js 与生产部署	Pilot có fallback 具备回退的试点

> **Bước nên làm đầu tiên:** Dựng Qwen nguyên bản và benchmark trên 500 câu chuẩn trước khi đầu tư vào fine-tune.
>
> **首要步骤：** 先部署原始 Qwen，并在 500 条标准句子上进行基准测试，再决定是否投入微调。

Ghi chú: VRAM, hyperparameter và thời gian trong tài liệu là điểm khởi đầu thực hành; cần benchmark trên hạ tầng và dữ liệu thực tế. 说明：文中的显存、超参数与时间仅为实践起点，最终应基于实际基础设施与数据进行基准测试。